Python Dictionaries Notes

Based on Chai Types Practice Session

1. Creating a Dictionary

```
chai_types = {"masala": "spicy", "Ginger": "zesty", "Green": "mild"}
```

- Dictionaries use `curly braces {}` with `key: value` pairs
- Keys and values can be strings, numbers, or other types
- Keys are case-sensitive — "Ginger" and "ginger" are different keys
- Duplicate keys are not allowed — the last one overwrites earlier ones

2. Accessing Values

.get()

```
chai_types.get("Ginger")    # Returns: 'zesty'
chai_types.get("ginger")    # Returns: None (key not found, no error)
```

- `.get(key)` safely retrieves a value — returns `None` if the key doesn't exist
- Key must be a string in quotes — without quotes Python looks for a variable

💡 *chai_types.get("ginger") returns nothing because the key is "Ginger" (capital G) — dictionaries are case-sensitive*

💡 *Error encountered: NameError: name 'Ginger' is not defined — fixed by wrapping in quotes: .get("Ginger")*

3. Looping Through a Dictionary

Keys only

```
for chai in chai_types:
    print(chai)
# Output: masala / Ginger / Green
```

- Looping over a dictionary directly iterates over its keys

Keys + Values via .get()

```
for chai in chai_types:
    print(chai, chai_types.get(chai))
# Output: masala spicy / Ginger zesty / Green mild
```

Keys + Values via .items()

```
for key, value in chai_types.items():
    print(key, value)
# Output: masala spicy / Ginger zesty / Green mild
```

- `.items()` returns each key-value pair as a tuple — cleaner than using .get() inside the loop

4. Checking Membership

```
if "masala" in chai_types:
    print("i have masala chai")
# Output: i have masala chai
```

- Use `in` to check if a key exists in the dictionary
- Variable names cannot have spaces — `chai types` is invalid; use `chai_types`

Error encountered: SyntaxError: invalid syntax — caused by a space in the variable name chai types

5. Adding & Removing Items

Adding a new key

```
chai_types["earl grey"] = "citrus"
# Result: {'masala': 'spicy', 'Green': 'mild', 'earl grey': 'citrus'}
```

- Assign a value to a new key using `dict["key"] = value`

.pop(key)

```
chai_types.pop("Ginger")   # Returns and removes: 'zesty'
```

- `.pop(key)` removes the specified key-value pair and returns the value

.popitem()

```
chai_types.popitem()   # Removes and returns the last inserted pair
# Returns: ('earl grey', 'citrus')
```

- `.popitem()` removes and returns the most recently added key-value pair

del keyword

```
del chai_types["Green"]
# Removes the 'Green' key entirely
```

- `del dict[key]` deletes a key-value pair without returning the value

6. Other Useful Operations

len()

```
print(len(chai_types))    # Returns: 3
```

- `len(dict)` returns the number of key-value pairs in the dictionary

.copy()

```
chai_types_copy = chai_types.copy()
```

- `.copy()` creates a shallow copy — changes to the copy don't affect the original

.clear()

```
squared_num.clear()
# Result: {}
```

- `.clear()` removes all items, leaving an empty dictionary

7. Dictionary Comprehension

```
squared_num = {x: x**2 for x in range(6)}
# Result: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

- Syntax: `{key: value for item in iterable}`
- A compact way to build a dictionary — similar to list comprehension but with key-value pairs

8. dict.fromkeys()

```
keys = ["masla", "Ginger", "lemon"]
default_value = "delicious"
new_dict = dict.fromkeys(keys, default_value)
# Result: {'masla': 'delicious', 'Ginger': 'delicious', 'lemon': 'delicious'}
```

- `dict.fromkeys(keys, value)` creates a new dictionary from a list of keys, all assigned the same default value
- Useful for initializing a dictionary with placeholder values

9. Common Errors Encountered

```
NameError: name 'Ginger' is not defined
```

Key was written without quotes. Dictionary keys must be strings in quotes: .get("Ginger"), not .get(Ginger).

```
SyntaxError: invalid syntax (chai types)
```

Variable names cannot contain spaces. Use underscores: chai_types instead of chai types.

10. Quick Reference

Operation	Example	Result
Create dict	`{"key": "value"}`	`New dictionary`
Access value	`.get("Ginger")`	`'zesty' or None`
Loop keys	`for k in dict:`	`Iterates keys`
Loop key+val	`for k, v in dict.items():`	`Both at once`
Check key	`"masala" in dict`	`True / False`
Add item	`dict["new"] = "val"`	`Adds key-value`
Remove by key	`.pop("key")`	`Returns value`
Remove last	`.popitem()`	`Returns pair`
Delete key	`del dict["key"]`	`No return`
Count items	`len(dict)`	`Integer`
Copy dict	`dict.copy()`	`Shallow copy`
Clear dict	`dict.clear()`	`{}`
Comprehension	`{x: x**2 for x in range(6)}`	`{0:0, 1:1, ...}`
From keys	`dict.fromkeys(keys, val)`	`All same value`